

Численные методы одномерной минимизации с использованием производной

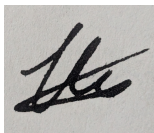
Копылов Иван Александрович

Москва 2026

Лабораторная работа №2

Вариант 10

Копылов Иван Александрович, БИВТ-24-9



НИТУ МИСИС

Лычёв Андрей Владимирович

1 Цель работы

Приобретение практических навыков для решения задач одномерной минимизации численными методами с использованием производной.

2 Задача

Требуется найти безусловный минимум функции одной переменной $y = f(x)$ на отрезке $[a, b]$, где функция является выпуклой и дифференцируемой. Необходимо найти такую точку $x^* \in [a, b]$, что

$$f(x^*) = \min_{x \in [a, b]} f(x).$$

Согласно варианту 10:

$$f(x) = 5x^2 + \frac{1}{x}, \quad a = 0.2, \quad b = 2, \quad \varepsilon = 0.0002.$$

Для поиска минимума реализованы два метода:

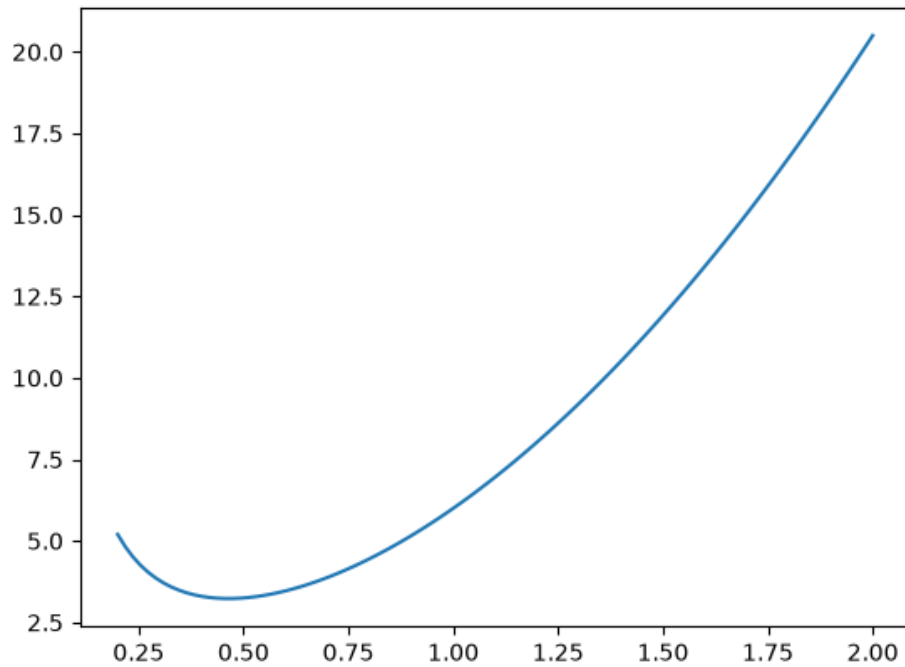
- *Метод 1* — метод средней точки;
- *Метод 2* — метод Брента.

3 GenAI

Использованные инструменты GenAI:

1. OpenAI — GPT-5. Использовался для оформления отчёта по данным из программного кода.

4 Графическое представление функции на заданном интервале



5 Аналитическое нахождение минимума

Найдём производные функции:

$$f'(x) = \left(5x^2 + \frac{1}{x}\right)' = 10x - \frac{1}{x^2},$$

$$f''(x) = 10 + \frac{2}{x^3}.$$

Необходимое условие экстремума:

$$10x - \frac{1}{x^2} = 0, \quad 10x^3 = 1, \quad x^* = \sqrt[3]{0.1} \approx 0.4641588834.$$

Так как

$$f''(x^*) = 10 + \frac{2}{(x^*)^3} = 30 > 0,$$

точка x^* является точкой минимума.

Значение функции в точке минимума:

$$f(x^*) = 5(x^*)^2 + \frac{1}{x^*} \approx 3.2316520350.$$

6 Листинг программы

```
import math
import matplotlib.pyplot as plt
import numpy as np

def f_base(x):
    ans = 5 * x**2 + 1 / x
    return ans

def d_base(x):
    ans = 10*x - 1/x**2
    return ans

# graph
xs = np.linspace(0.2, 2, 100)
fs = np.array(list(map(f_base, xs)))

plt.plot(xs, fs)
plt.show()

# Method 1: midpoint method
def method_1(a, b, eps):
    iter_c = 0
    f_call_c = 0

    def f(x):
        nonlocal f_call_c
        f_call_c += 1
        return f_base(x)

    def d(x):
        nonlocal f_call_c
        f_call_c += 1
        return d_base(x)

    x = (a+b)/2
    d_x = d(x)

    while abs(a - b) > eps or abs(d_x) > eps:
        iter_c += 1
        if d_x > 0:
            b = x
        else:
            a = x
```

```

        x = (a+b)/2
        d_x = d(x)

    return (x, f(x), iter_c, f_call_c)

# Method 2: Brent method
def method_2(a, b, eps):
    iter_c = 0
    f_call_c = 0

    def f(x):
        nonlocal f_call_c
        f_call_c += 1
        return f_base(x)

    f_a = f(a)
    f_b = f(b)
    x = a + (3 - math.sqrt(5)) / 2 * (b - a)
    f_x = f(x)
    e = 0

    while max(x - a, b - x) > 2 * eps:
        iter_c += 1
        m = (a + b) / 2
        p = math.pow(b - x, 2) * (f_a - f_x) - math.pow(x - a, 2) * (f_b
        - f_x)
        q = 2 * ((b - x)*(f_a - f_x) + (x - a)*(f_b - f_x))
        s = x + p/q

        if (q == 0 and abs(e) < eps and (s < a or s > b)) or abs(p/q) >
        abs(e)/2:
            if x >= m:
                u = (math.sqrt(5) - 1) / 2 * x + (3 - math.sqrt(5)) / 2
            * a
                f_u = f(u)
            else:
                u = (math.sqrt(5) - 1) / 2 * x + (3 - math.sqrt(5)) / 2
            * b
                f_u = f(u)
            else:
                u = x + p/q
                f_u = f(u)
                if (abs(u - x) < 2*eps and u - a < 2*eps) or (b - u) < 2*eps
                :
                    u = x + eps * np.sign(p/q)

```

```

        f_u = f(u)

    if f_u <= f_x:
        if u < x:
            b = x
        else:
            a = x
        x = u
        f_x = f_u
    else:
        if u < x:
            a = u
        else:
            b = u

    return (x, f_x, iter_c, f_call_c)

a = 0.2
b = 2
eps = 0.0002

(xm_1, f_xm_1, iter_c_1, f_call_c_1) = method_1(a, b, eps)
print(iter_c_1, f_call_c_1, xm_1, f_xm_1)

(xm_2, f_xm_2, iter_c_2, f_call_c_2) = method_2(a, b, eps)
print(iter_c_2, f_call_c_2, xm_2, f_xm_2)

```

7 Результаты вычислений

В программе были получены следующие результаты:

Название метода	Вычислений	Итераций	Результат вычислений	Погрешность по x
Аналитическое значение	~	~	$x^* \approx 0.4641588834, f(x^*) \approx 3.2316520350$	~
Метод средней точки	18	16	$x = 0.4641525269, f(x) = 3.2316520357$	0.0000063565
Метод Брента	20	17	$x = 0.4642476346, f(x) = 3.2316521532$	0.0000887513

8 Сравнительная характеристика методов

Метод средней точки выполнил 16 итераций и 18 вычислений функции или производной. После исправления условия остановки метод дал точное приближение к аналитическому минимуму: погрешность по координате составила примерно 0.00000635.

Метод Брента потребовал 17 итераций и 20 вычислений функции. Найденная точка $x = 0.4642476346$ также отличается от аналитического значения менее чем на 0.0001, однако в данном эксперименте она оказалась немного менее точной по координате, чем результат метода средней точки.

Таким образом, в данном вычислительном эксперименте оба метода достигли заданной точности, но метод средней точки оказался немного эффективнее: он потребовал меньше итераций и вычислений, а также дал меньшую погрешность по x .

9 Выводы

В ходе лабораторной работы была исследована функция $f(x) = 5x^2 + 1/x$ на отрезке $[0.2, 2]$. Аналитически установлено, что минимум достигается при

$$x^* \approx 0.4641588834, \quad f(x^*) \approx 3.2316520350.$$

По данным программы оба метода дали результаты, близкие к аналитическому минимуму. Метод средней точки показал наилучший результат в этом эксперименте: при меньшем числе вычислений он получил более точное значение координаты минимума. Метод Брента также подтвердил сходимость к минимуму, но потребовал немного больше вычислений.